

BigQuery ML (BQML) Handbook

Training, Evaluating, and Explaining Machine Learning Models in SQL

This handbook explains BigQuery ML in depth — what it is, why it exists, the full model lifecycle, every major model type, and explainability/anomaly detection — using one running HR analytics scenario (100 employees, 8 departments, 1,200 monthly attendance records) to ground every concept in a concrete question and a verifiable answer.

Problem statement: IndexCorp's HR team keeps asking questions that sound like machine learning problems — predicting salary, spotting high performers, segmenting the workforce, forecasting attendance — but the data science team is small, and every request today means exporting data, writing Python, and shipping results back. This handbook shows how to answer all of it with SQL alone, inside BigQuery, with no data ever leaving the warehouse.

1. What Is BigQuery ML, and Why Does It Exist?

BigQuery ML lets you create, train, evaluate, and use machine learning models using standard SQL, directly against tables already in BigQuery — no data export, no separate ML infrastructure, no Python environment required to get a working model.

1.1 The Case For It

Traditional ML workflow	BQML workflow
Export data from the warehouse to a notebook/training environment	Data never leaves BigQuery
Requires Python, a ML framework (scikit-learn/TensorFlow), and an ML engineer	Requires SQL — the skill most analysts already have
Separate compute/infrastructure to manage for training	Training runs as a BigQuery job, on the same infrastructure as your queries
Serving predictions often needs a separate deployment step	ML.PREDICT is just another SQL query against the trained model

The honest trade-off: BQML isn't a replacement for a dedicated ML platform on every problem — deep learning at scale, custom architectures, and cutting-edge research still belong in a framework like PyTorch/Vertex AI custom training. BQML's sweet spot is exactly the IndexCorp scenario: well-understood problems (regression, classification, clustering, forecasting) where SQL fluency, not ML specialization, is the actual bottleneck.

1.2 The Model Lifecycle — the Same Three Steps, Every Time

Step	SQL	Purpose
Train	CREATE MODEL ... OPTIONS(model_type=...) AS SELECT ...	Fit the model on your data
Evaluate	SELECT * FROM ML.EVALUATE(MODEL ...)	Check quality with the metrics appropriate to the model type
Use	ML.PREDICT / ML.FORECAST / ML.DETECT_ANOMALIES	Get results — predictions, forecasts, or anomaly flags

Every one of this handbook's seven HR questions follows exactly this shape — only the model_type and which "use" function applies changes.

2. Model Types at a Glance

Model type	Learning style	Answers questions like...	Used for
LINEAR_REG	Supervised (regression)	"What should this continuous number be?"	Salary benchmarking (Q1)
LOGISTIC_REG	Supervised (classification)	"Which category does this fall into?" (binary or multiclass)	High-performer prediction (Q2)
BOOSTED_TREE_CLASSIFIER/REGRESSOR	Supervised	Same as above, with a more flexible, often more accurate algorithm	Comparing against logistic regression (Q3)
KMEANS	Unsupervised (clustering)	"What natural groups exist in this data?"	Employee segmentation (Q5)
ARIMA_PLUS	Supervised (time series)	"What will this metric look like in the future?"	Attendance forecasting (Q7)
AUTOENCODER / PCA	Unsupervised	Dimensionality reduction; also usable for anomaly detection	Not used directly in this lab, but the same anomaly-detection function applies
MATRIX_FACTORIZATION	Supervised (recommendation)	"What would this user rate/prefer?"	Not a natural fit for HR data — mentioned for completeness

3. The Dataset & the Reusable Feature View

Three tables — departments (8 rows), employees (100 rows), and attendance_monthly (1,200 rows: 100 employees × 12 months) — joined once into a single feature view that every model in this handbook trains against.

```
CREATE OR REPLACE VIEW `hr_bqml_lab.employee_features_view` AS
SELECT
  e.employee_id, e.gender, e.state, d.department_name, e.designation,
  DATE_DIFF(CURRENT_DATE(), e.date_of_joining, DAY) AS tenure_days,
  e.monthly_salary_inr,
  ROUND(AVG(a.days_present), 2) AS avg_days_present,
  ROUND(AVG(a.leaves_taken), 2) AS avg_leaves_taken,
  ROUND(AVG(a.performance_rating), 2) AS avg_performance_rating,
  (AVG(a.performance_rating) >= 4.0) AS high_performer
FROM `hr_bqml_lab.employees` e
JOIN `hr_bqml_lab.departments` d ON e.department_id = d.department_id
JOIN `hr_bqml_lab.attendance_monthly` a ON e.employee_id = a.employee_id
GROUP BY e.employee_id, e.gender, e.state, d.department_name, e.designation,
         e.date_of_joining, e.monthly_salary_inr;
```

high_performer is a derived label, not raw data: the 4.0 threshold is a business decision you'd make with the HR team — worth stating explicitly, since students should understand every supervised model's label came from somewhere, not from the data itself.

Check it in the console: hr_bqml_lab dataset → employee_features_view's Details tab shows Table type: VIEW, confirming no data is duplicated by this join.

4. Question 1 — Salary Benchmarking (Linear Regression)

The problem: is a given employee's salary in line with what their role, department, and tenure would predict?

```
CREATE OR REPLACE MODEL `hr_bqml_lab.salary_model`  
OPTIONS(model_type='linear_reg', input_label_cols=['monthly_salary_inr']) AS  
SELECT department_name, designation, state, gender, tenure_days, monthly_salary_inr  
FROM `hr_bqml_lab.employee_features_view`;  
  
SELECT * FROM ML.EVALUATE(MODEL `hr_bqml_lab.salary_model`);  
  
SELECT employee_id, monthly_salary_inr AS actual, predicted_monthly_salary_inr AS predicted  
FROM ML.PREDICT(MODEL `hr_bqml_lab.salary_model`, (SELECT * FROM `hr_bqml_lab.employee_features_view`)  
)  
ORDER BY ABS(monthly_salary_inr - predicted_monthly_salary_inr) DESC;
```

Regression metrics to read: `r2_score` (how much variance the model explains — closer to 1 is better) and `mean_absolute_error` (average prediction error, in the same units as the label — rupees here).

Check it in the console: Model's Evaluation tab shows `r2_score`/`mean_absolute_error` as summary cards; Training tab shows the loss curve.

5. Questions 2 & 3 — High-Performer Prediction, Two Algorithms Compared

The problem: proactively identify high performers, and check whether a more flexible algorithm actually improves on a simple baseline for this specific task.

```
-- Baseline  
CREATE OR REPLACE MODEL `hr_bqml_lab.high_performer_logistic_model`  
OPTIONS(model_type='logistic_reg', input_label_cols=['high_performer']) AS ...  
  
-- Comparison, same features/label  
CREATE OR REPLACE MODEL `hr_bqml_lab.high_performer_boosted_tree_model`  
OPTIONS(model_type='boosted_tree_classifier', input_label_cols=['high_performer']) AS ...
```

Classification metrics to read: `precision` (of predicted positives, how many were actually right), `recall` (of actual positives, how many did the model catch), and `roc_auc` (overall discrimination ability, 0.5=random, 1.0=perfect).

A genuinely useful lesson from comparing them: on a small dataset (100 employees), a more flexible model isn't automatically better — it can overfit where a simpler model generalizes fine. The gap typically favors boosted trees more clearly only at much larger scale, where non-linear feature interactions actually exist to be captured.

Check it in the console: Both models listed side by side in `hr_bqml_lab`; open each Evaluation tab to compare ROC curves visually.

6. Question 4 — Explainability

The problem: a raw prediction score doesn't answer "why" — a manager needs to know which specific factors drove a given prediction, for a specific employee.

6.1 Global Feature Importance

```
SELECT * FROM ML.FEATURE_IMPORTANCE(MODEL `hr_bqml_lab.high_performer_boosted_tree_model`);
```

Available for tree-based models (boosted trees, random forest) — ranks features by how much they contributed to the model's decisions *overall*, across every employee.

6.2 Per-Prediction Explanation

```
SELECT employee_id, predicted_high_performer, top_feature_attributions
FROM ML.EXPLAIN_PREDICT(MODEL `hr_bqml_lab.high_performer_boosted_tree_model`, (
  SELECT * FROM `hr_bqml_lab.employee_features_view` LIMIT 5
), STRUCT(3 AS top_k_features));
```

Global vs. per-prediction — a distinction worth stating explicitly: ML.FEATURE_IMPORTANCE answers "what matters most across all employees"; ML.EXPLAIN_PREDICT answers "what mattered most for this one employee's specific prediction" — the latter is what actually answers a manager's "why this person" question.

Check it in the console: No dedicated console visualization for per-prediction explanations — query-only, best explored in the SQL workspace results grid.

7. Question 5 — Employee Segmentation (K-Means)

The problem: rather than guessing at employee segments, let unsupervised learning find natural groupings — no label needed, since there's no predetermined "correct" segment.

```
CREATE OR REPLACE MODEL `hr_bqml_lab.employee_segments_model`
OPTIONS(model_type='kmeans', num_clusters=3) AS
SELECT avg_days_present, avg_leaves_taken, avg_performance_rating
FROM `hr_bqml_lab.employee_features_view`;

SELECT centroid_id, feature, numerical_value FROM ML.CENTROIDS(MODEL `hr_bqml_lab.employee_segments_model`);
```

Choosing num_clusters: 3 is a reasonable starting guess for a small dataset; a real project would compare the Davies-Bouldin index across a few candidate values (available in ML.EVALUATE's output for KMEANS models) before settling on a final count.

Check it in the console: Model's Evaluation tab shows a cluster visualization plotting segments across two of the three features.

8. Question 6 — Anomaly Detection

The problem: rather than a human manually scanning every employee's attendance pattern for outliers, reuse the K-Means model to flag anyone unusually far from any cluster.

```
SELECT employee_id, is_anomaly, normalized_distance, centroid_id
FROM ML.DETECT_ANOMALIES(
  MODEL `hr_bqml_lab.employee_segments_model`,
  STRUCT(0.1 AS contamination),
  (SELECT employee_id, avg_days_present, avg_leaves_taken, avg_performance_rating
   FROM `hr_bqml_lab.employee_features_view`)
)
ORDER BY normalized_distance DESC;
```

How this works for K-Means specifically: each point's distance to its nearest centroid is normalized by that cluster's typical spread (standard deviation of distances within the cluster). contamination sets what fraction of the population is expected to be anomalous (0.1 = roughly the most unusual 10%). The same function also supports AUTOENCODER and PCA models (reconstruction-error based) and ARIMA_PLUS/ARIMA_PLUS_XREG (confidence-interval based, for time series).

Check it in the console: No separate anomaly-detection tab — the underlying model is the same employee_segments_model visible in hr_bqml_lab; results are query-only.

9. Question 7 — Time Series Forecasting (ARIMA_PLUS)

The problem: workforce planning wants a data-driven expectation for the next quarter, not a guess.

```
CREATE OR REPLACE MODEL `hr_bqml_lab.attendance_forecast_model`
OPTIONS(model_type='ARIMA_PLUS', time_series_timestamp_col='month_date',
        time_series_data_col='avg_days_present', auto_arima=TRUE) AS
SELECT month_date, AVG(days_present) AS avg_days_present
FROM `hr_bqml_lab.attendance_monthly` GROUP BY month_date;

SELECT * FROM ML.ARIMA_EVALUATE(MODEL `hr_bqml_lab.attendance_forecast_model`);

SELECT forecast_timestamp, forecast_value, prediction_interval_lower_bound, prediction_interval_upper_bound
FROM ML.FORECAST(MODEL `hr_bqml_lab.attendance_forecast_model`, STRUCT(3 AS horizon, 0.95 AS confidence_level));
```

Honest caveat on data volume: 12 months of history demonstrates the mechanism but is well short of the multiple full seasonal cycles ARIMA_PLUS really wants for a trustworthy forecast — treat any specific number from this exercise as illustrative, not something to act on directly.

Check it in the console: Model's Evaluation tab plots the forecast with confidence-interval bands directly — no charting code needed.

10. Cleanup

Deleting the dataset removes every table, view, and all five models created across this handbook's seven questions, in one operation.

```
bq_client.delete_dataset(f'{PROJECT_ID}.hr_bqml_lab', delete_contents=True, not_found_ok=True)
```

Check it in the console: BigQuery > SQL workspace > Explorer → the hr_bqml_lab dataset should no longer be listed under your project at all.

This handbook's companion notebook: bqml_hr_analytics_lab.ipynb builds and runs every model above end-to-end against your own project, using the same GitHub-hosted CSVs, with the same problem-to-solution structure and full guarded cleanup.